

%Qcache% in the server status to see how well the query cache is being used.

- **tmp_table_size:** MySQL sometimes needs to create temporary tables automatically, depending on the query. When the result set grows larger than the size specified here, the temporary table is converted to a disk-based temporary table, which results in a loss in performance. Keep this size as large as possible, considering the available DRAM and other applications running on the site. In the server status, observe *created_tmp_disk_tables*.
- **max-connections:** The maximum number of simultaneous connections. This is limited by CPU power and the available DRAM.
- **sort_buffer:** This should be set to a high value if your queries involve a lot of sorting. This seems to have some negative effect on performance as well, so do a few test cases before and after changing the value.
- **thread_cache:** If you have a lot of very-short-duration connections, increase this value till the *threads_created* value stops increasing in the server status, to reduce CPU usage.
- **long_query_time:** This is the time a query will be allowed to run before it is considered slow and logged in *slowlog*. Keep this value small—say 5s; the default value is 10s.
- **slow_query_log_file:** This is the path to the slow query log file. MySQL 5.5 has a new option, *log-queries-not-using-indexes*. If specified, it will send only slow queries that are not using indexes to the slow query log.

Tips for designing efficient tables and queries

Although I have talked a lot about options and optimisations, the actual optimisation of MySQL is done by properly designing tables and the database. Also, it depends on how efficient your queries are. Generally, one query can be executed in multiple ways, and each way has its own advantages and disadvantages. We need to choose a query that suits our environment and isn't costly on the performance side. Here are some points:

- Keep the design simple; split your columns across tables. Columns you don't access often should be put in a different table, to avoid memory wastage when the table is cached. You can link data across tables using foreign keys, which are supported only in InnoDB, and not in MyISAM.
- Use MyISAM if you are going to read a lot and write less—and InnoDB otherwise. There are some absolutely astounding results in this aspect, and you should switch engines after proper testing.
- The columns frequently used in *WHERE* statements and in filtering results should be indexed. Indexes point to the data location on the disk. If indexes are not used,

MySQL has to scan the whole table to find the required row, which is very costly in terms of performance.

- Your tables should conform to normal form rules, have normalised data, and use efficient column types.
- Keep queries simple and short. Longer queries take more time to execute. Use application-level caching.
- Turn on slow query logging in *my.cnf* by setting *long_query_time* and *slow_query_log_file*.
- Use *EXPLAIN* and *ANALYZE* to see what a slow query does, and how long it takes. This can give you a lot of hints for optimising it.
- Run *OPTIMIZE TABLE* frequently. But be aware: this locks the table, so run it only when MySQL traffic is low. This basically rearranges disk files, reclaiming disk space consumed by deleted rows.

Since MySQL uses a lot of disk, actual performance is pretty much limited by disk performance, rotation speed, etc. Obviously, you need a high-speed disk. Another optimisation tip would be to use a file system that supports compression on the MySQL data partition, to reduce disk seeks; this is currently supported in ZFS (FreeBSD) and Btrfs. The latter is under development as of now, and not recommended for production use; try it only after proper backups.

Using RAID striping with a stripe size of 1 MB or so can help a lot, since the data is split across two disks, and reading from two disks is obviously faster than reading from one disk sequentially.

There's another possibility for speeding up disk performance, which is not actually tested: using SquashFS and AUFS. I have never tried it for a MySQL database directory but have used it on my system, and it does speed up things at the cost of CPU usage. What you basically do is compress */usr* into a *squashfs* file—say, */squashed/usr.sqfs*. Then, you mount *usr.sqfs* at */squashed/usr/ro* (loop, readonly). Using AUFS, you can mount */usr* and specify read (*/squashed/usr/ro*) and write (*/squashed/usr/rw*) locations as its branches. The only problem with this method is that you have to re-squash whenever large changes are made, in order to ensure performance. This is probably not feasible with MySQL in cases where the database will have a lot of writes. More information on this squashing method can be obtained from the Gentoo Linux forum thread at <http://j.mp/nGMPvs>. Please note—I'm not responsible for any loss. Back up before trying any tricks. :-)



By: Nileshe Govindarajan

The author has been using Linux for more than four years. He keeps taking up small programming projects and works on server administration. He can be reached at contact@nileshgr.com or @nileshgr on Twitter/identi.ca